



Lab Title: OWASP Penetration Testing Methodology

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS105 – Web Application Penetration Testing Methodology

Instructor Name: AMMAR AMINU

Date of Submission: 9/04/2026

Executive Summary

This assessment focused on testing the security of the OWASP Juice Shop web application by simulating real-world attack scenarios. The goal was to identify weaknesses an attacker could exploit.

During the testing, several vulnerabilities were discovered, including injection flaws, broken authentication, and improper input handling. These issues could allow unauthorized access, data exposure, and manipulation of application behavior.

From a business perspective, these risks could lead to loss of customer trust, data breaches, and reputational damage if not addressed. Immediate attention is recommended to fix the identified issues and improve overall security posture.

Lab Objectives

The objective of this lab was to simulate a penetration testing engagement against the OWASP Juice Shop application.

The aim was to:

- Understand how attackers explore web applications
- Identify common web vulnerabilities based on OWASP standards
- Perform basic exploitation techniques
- Document findings in a professional format

Tools Used

The following tools were used during the assessment:

- **Web Browser (Chrome/Firefox)** – for interacting with the application
- **Burp Suite** – for intercepting and modifying HTTP requests
- **CyberChef** – for encoding payloads (e.g., HTML, hex)
- **OWASP ZAP** (optional) – for automated scanning
- Developer tools (browser console & network tab)

Lab Environment

- Target Application: OWASP Juice Shop (hosted online)
- URL: <https://juice-shop.herokuapp.com>
- Attacker Machine: Personal computer (Windows OS with browser and tools installed)
- Network: Standard internet connection

The lab was conducted in a controlled and safe environment meant for learning and practice.

Lab Methodology

The testing followed a structured penetration testing approach:

1. **Reconnaissance & Mapping**
The application was explored to understand its structure, pages, and input points.
2. **Vulnerability Identification**
Potential weaknesses were identified using OWASP Top 10 as a guide.
3. **Testing & Exploitation**
Different attack techniques such as SQL injection and input manipulation were tested.
4. **Analysis**
The impact of each vulnerability was evaluated.
5. **Reporting**
All findings were documented clearly for both technical and non-technical audiences.

SECTION 1: Reconnaissance & Application Mapping

Task:

I access the Juice Shop application and explore it like an attacker.

Deliverable:

Page	Input field	Possible use	Link
Main page	Search bar	The search functionality is vulnerable to injection attacks due to	https://juice-shop.herokuapp.com/#/

		improper input sanitization. An attacker can inject malicious payloads such as SQL or JavaScript, leading to unauthorized data access or client-side code execution.	
Input Point	Login Form	Input forms can be exploited when user-supplied data is not properly validated, sanitized, or safely handled before being processed by the application. This allows attackers to inject malicious input that can lead to vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and command injection.	https://juice-shop.herokuapp.com/#/login

SECTION 2: Vulnerability Identification (OWASP-Based Thinking)

Task:

I identify at least 5 possible vulnerabilities in the application.

Deliverable:

Vulnerability	Location (Link)	Possible Impact	Remediation
SQL Injection	https://juice-shop.herokuapp.com/#/login	Unauthorized access to sensitive data, manipulation or deletion of records, and even full compromise of the database or underlying system.	Use parameterized queries or stored procedures. Never concatenate user input directly into SQL statements.
Time-Based Blind Injection	Burp Suite	The attacker infers database behavior by	Implement input validation and use

		sending payloads that delay responses (e.g., SLEEP()), allowing data extraction even when no error/output is shown.	prepared statements. Disable verbose error messages and monitor unusual delays.
Union-Based SQL Injection		The attacker uses UNION SELECT to combine malicious queries with legitimate ones, retrieving sensitive data such as user credentials from the database.	Restrict use of union in queries, sanitize inputs and apply inputs, and use content security policy
Cross-Site Scripting(XSS)	https://juice-shop.herokuapp.com/#/contact	Attacker injects malicious JavaScript that executes in users' browsers, leading to session hijacking, credential theft, or defacement.	Encode output before rendering in the browser, sanitize user input, and use Content Security Policy (CSP).
Error-Based SQLi	https://juice-shop.herokuapp.com/#/login	The attacker triggers database error to get information. Error messages reveal database structure (table, columns).	Suppress detailed error messages, use parameterized queries, and log errors securely without exposing database details.

SECTION 3: Practical Testing (Simulated Exploitation)

Target Application: OWASP Juice Shop

Testing Type: Simulated Penetration Testing

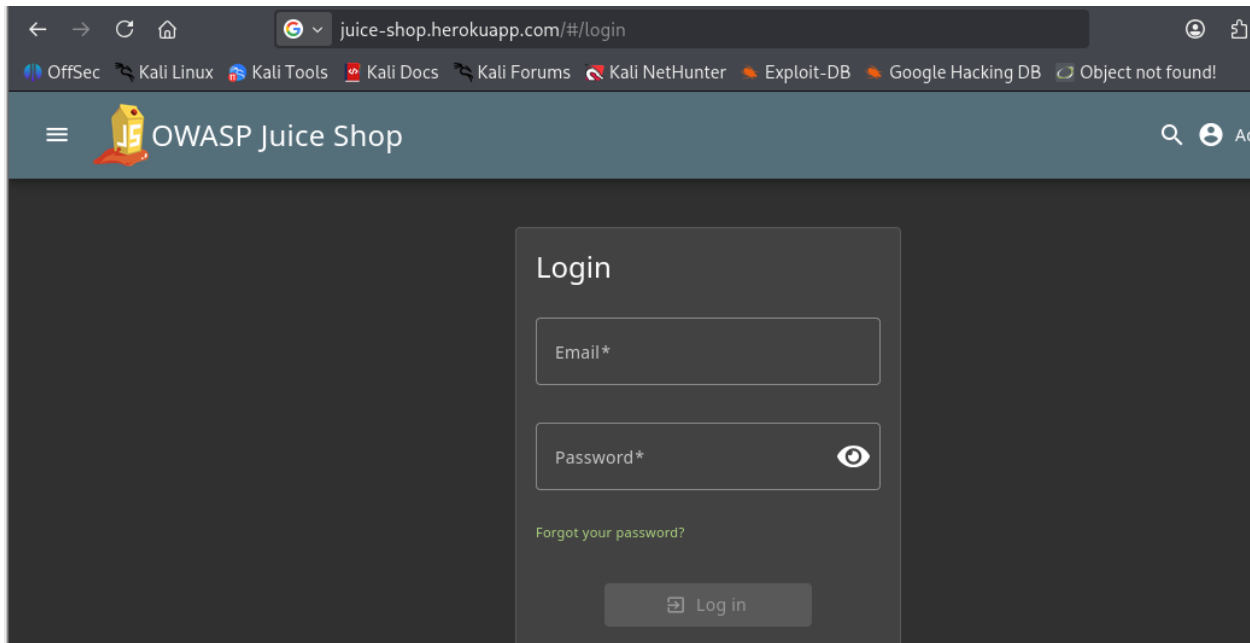
Objective: To identify and demonstrate exploitable vulnerabilities through controlled attack simulations.

1. SQL Injection (Authentication Bypass)

Objective: Bypass login authentication using SQL Injection.

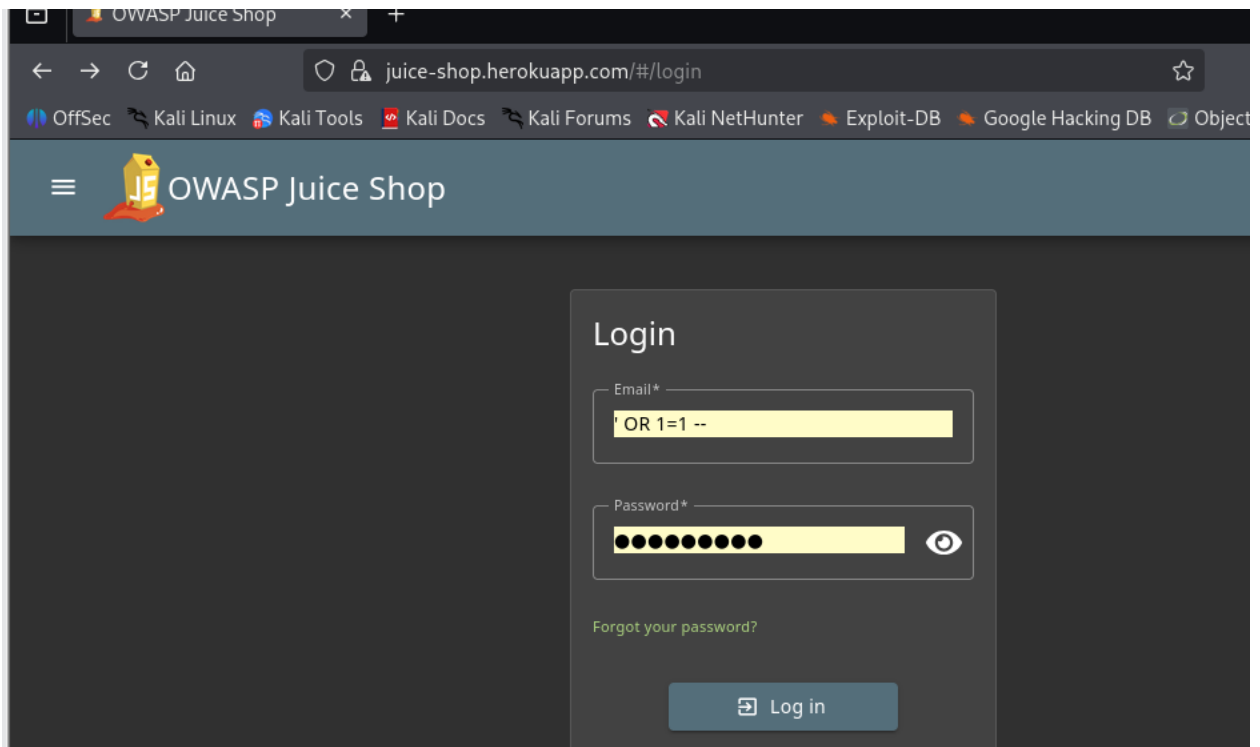
Steps to Reproduce:

1. Navigate to the login page.

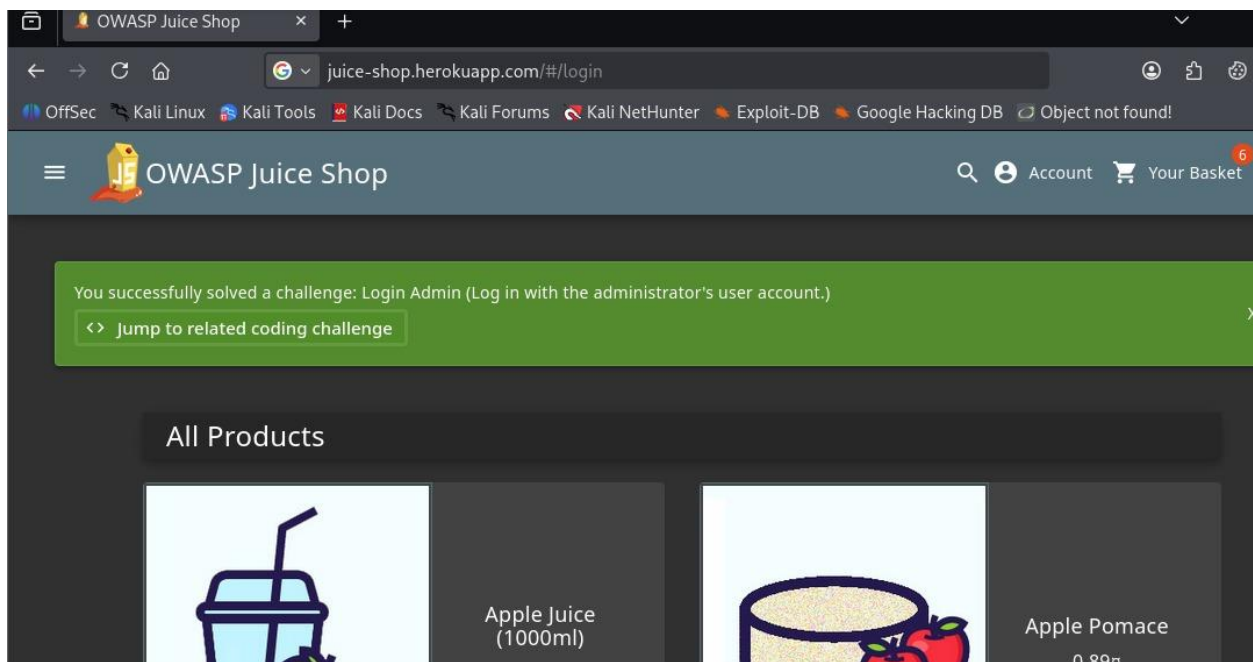


2. Enter the following payload in the email field:

' OR 1=1 –



3. Enter any value in the password field.
4. Click **Login**.



Expected Result:

- The application logs in without valid credentials.

Explanation:

- The payload manipulates the SQL query logic:

```
SELECT * FROM users WHERE email="" OR 1=1 -- AND password='...'
```

- 1=1 always evaluates to TRUE, granting access.

Impact:

- Unauthorized access to user accounts.
- Potential privilege escalation if admin accounts are targeted.

2. Error-Based SQL Injection

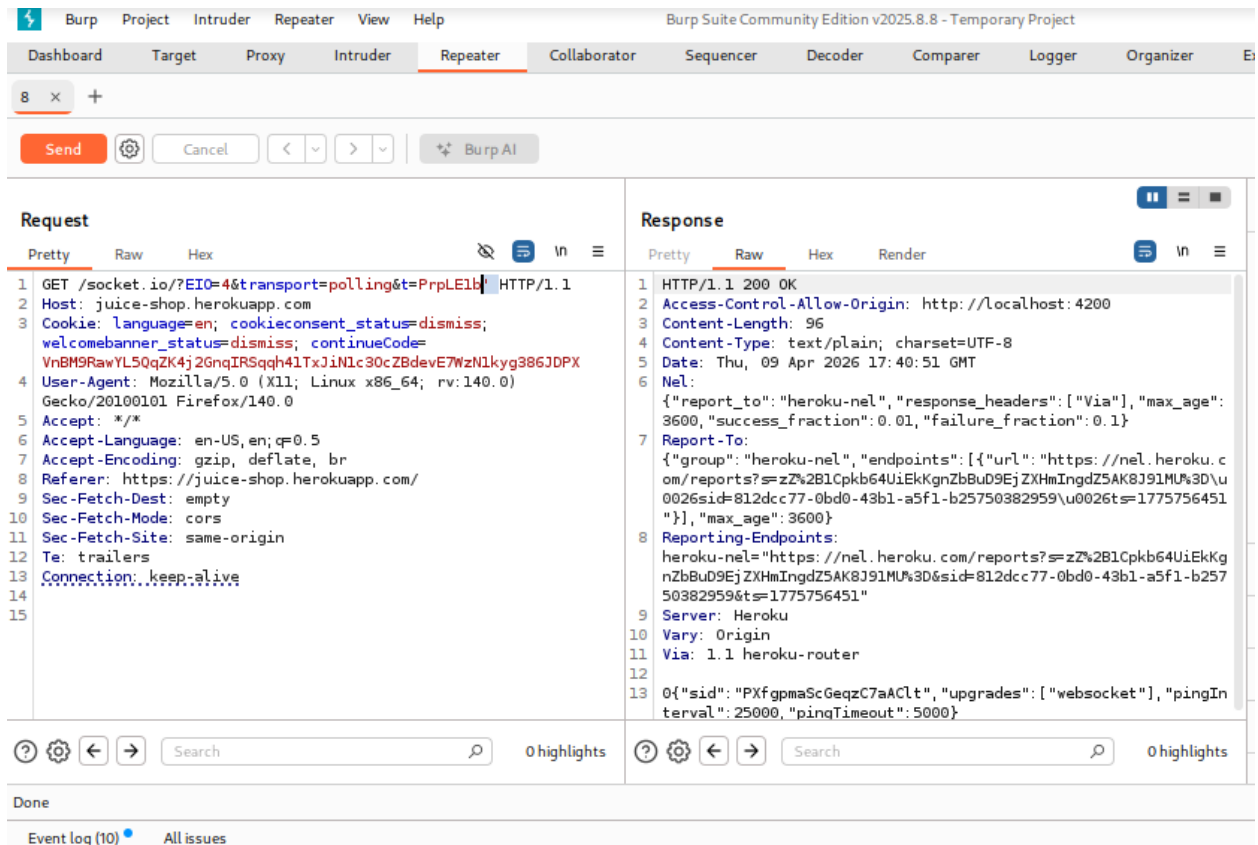
Objective: Extract database information via error messages.

Steps to Reproduce:

1. Navigate to burp suite to intercept the request
2. Enter:

'

3. Submit the request.



Expected Result:

- Application returns a database error message.

Explanation:

- Improper input handling causes SQL syntax errors.
- Error messages may reveal:
 - Table names
 - Column names
 - Database type

Impact:

- Information disclosure aiding further attacks.

3. Union-Based SQL Injection

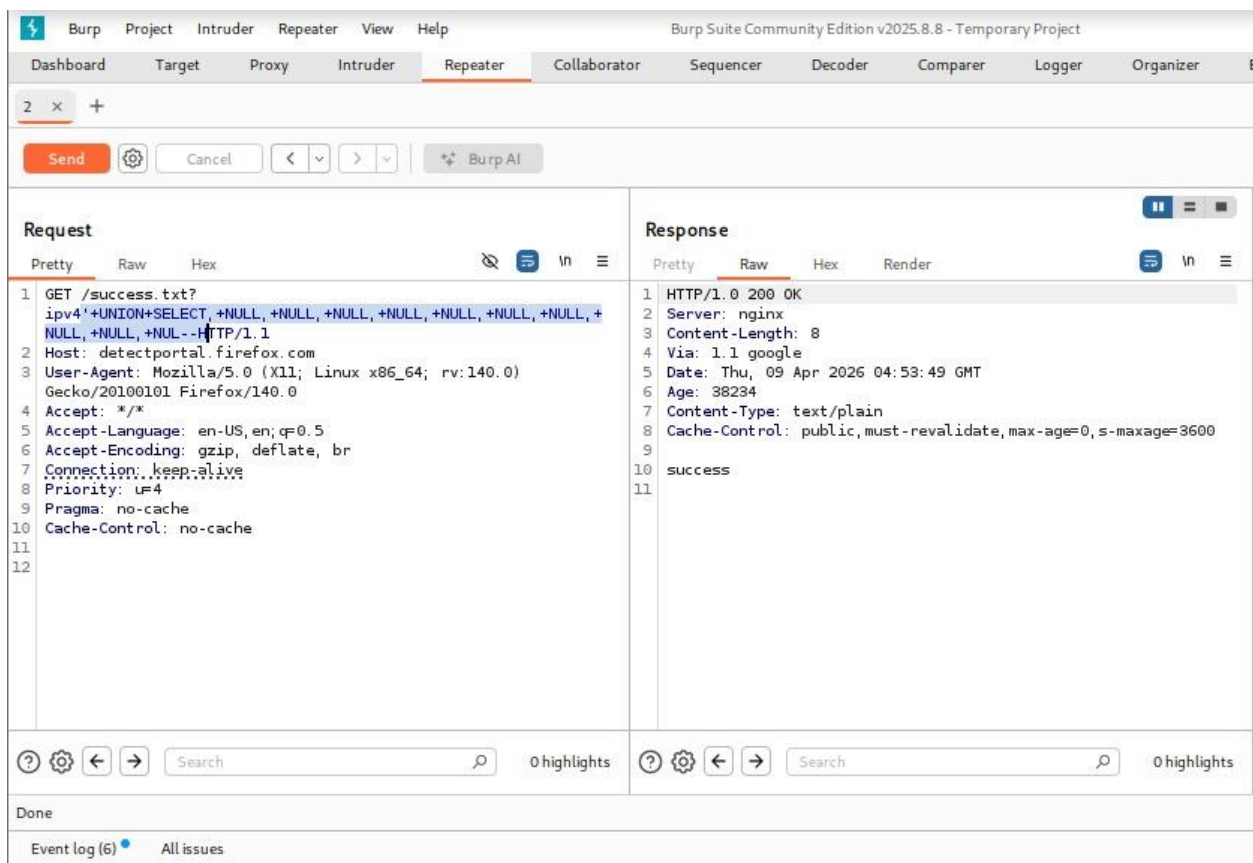
Objective: Retrieve hidden data from the database.

Steps to Reproduce:

1. Intercept the request using Burp Suite
2. Modify the query parameter:

' UNION SELECT NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL--

3. Send the request.



Expected Result:

- Application displays user credentials or additional data.

Explanation:

- UNION SELECT combines attacker query with original query.
- Allows extraction of sensitive data.

Impact:

- Exposure of credentials and sensitive records.

4. Time-Based Blind SQL Injection

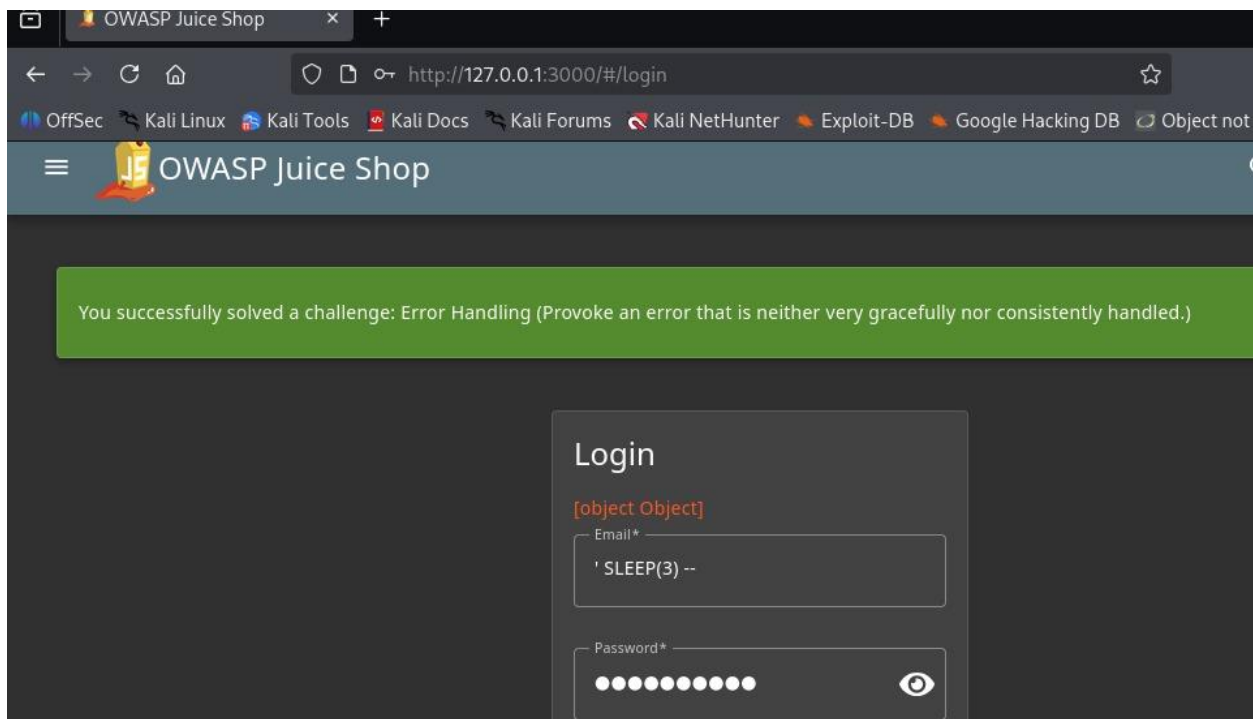
Objective: Confirm SQL injection vulnerability without visible output.

Steps to Reproduce:

1. Navigate to a vulnerable input field (e.g., search).
2. Enter:

' OR SLEEP(3) --

3. Submit the request and observe response time.



Expected Result:

- Application response is delayed by ~5 seconds.

Explanation:

- The database executes the delay function.
- Confirms injection even when no data is returned.

Impact:

- Attackers can extract data bit-by-bit using timing attacks.

5. Cross-Site Scripting (XSS)

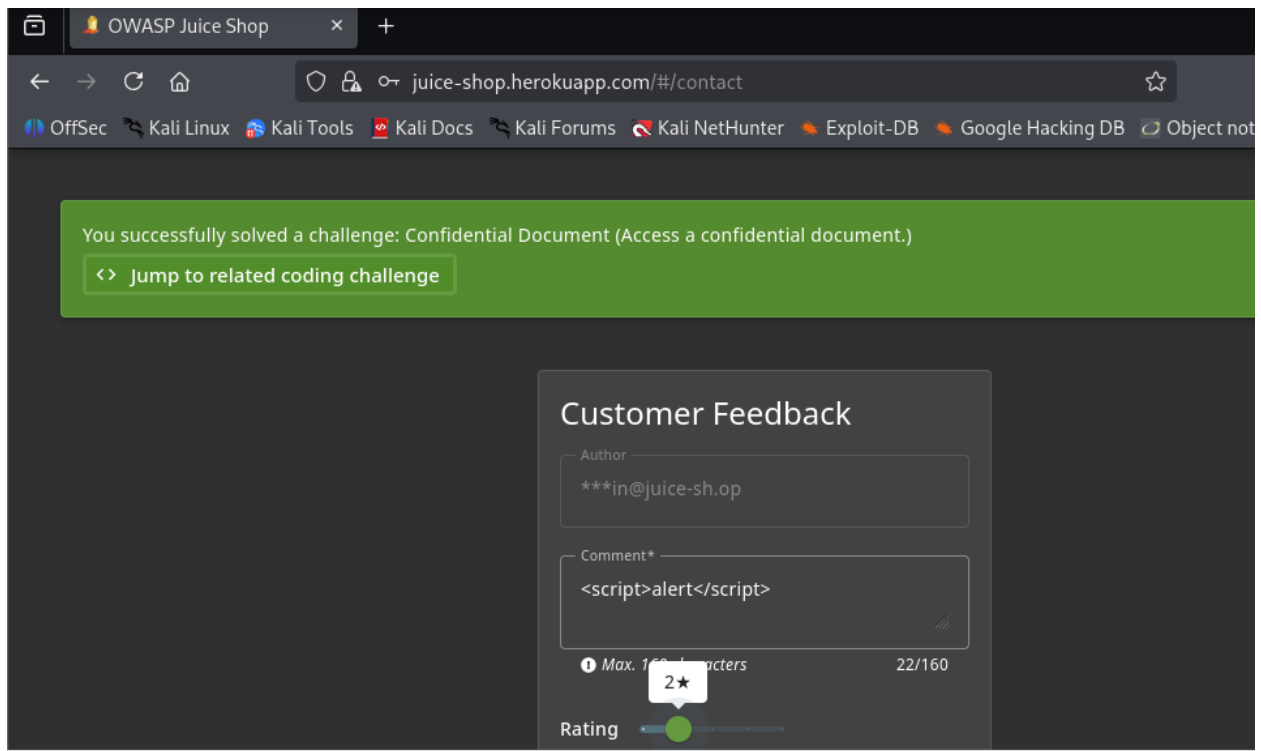
Objective: Execute malicious JavaScript in the browser.

Steps to Reproduce:

1. Navigate to the search or feedback form.
2. Enter:

<script>alert</script>

3. Submit the input.



Expected Result:

- A popup alert appears in the browser.

Explanation:

- Application fails to sanitize user input.
- Script executes in the victim's browser context.

Impact:

- Session hijacking
- Credential theft
- Malicious redirection

SECTION 4: Findings & Risk Analysis

ID	Vulnerability	Affected Endpoint	Payload	Evidence	Impact	Risk Level	Recommendation
01	SQL Injection	User Login	' OR 1=1 --	Login bypass observed ; user accessed without valid credentials	Unauthorized access to user accounts	High	Use parameterized queries and input validation
02	Cross-Site Scripting(XSS)	Search bar	<script>alert</script>	JavaScript alert executed in browser	Session hijacking , data theft	High	Encode output and validate input
03	Information Disclosure	Error response	'	SQL error message displayed	Reveals backend structure	Medium	Disable verbose error messages

04	Broken Authentication	User Login	' OR 1=1--	Login successful without valid credentials	Unauthorized access to user accounts	High	Implement parameterized queries and strong authentication controls
-----------	-----------------------	------------	------------	--	--------------------------------------	------	--

SECTION 5: Reporting (Executive vs Technical)

Part A: Executive Summary (Non-Technical)

- **Overview**

A security assessment was conducted on the web application to identify potential weaknesses that could be exploited by attackers. The testing focused on key areas of the application, including user authentication, input fields, and access to sensitive data.

- **What was tested**

The following components of the application were examined:

- Login and authentication system
- Search and input forms
- API endpoints and URL parameters
- Access control mechanisms (user roles and permissions)
- Error handling and system responses

The testing approach followed standard web application security practices based on **OWASP guidelines**.

- **Key Risks Identified**

The assessment revealed several security issues that may expose the application to attack:

- Weak authentication controls that could allow unauthorized access
- Improper handling of user input, making the system vulnerable to injection attacks
- Inadequate access control, allowing users to view or modify data that should be restricted
- Exposure of system information through error messages.

- **Business Impact**

If these issues are not addressed, they may lead to:

- Unauthorized access to user accounts
- Exposure of sensitive customer or business data
- Loss of user trust and reputational damage
- Potential financial loss due to system misuse or data breaches

Overall, the identified vulnerabilities increase the risk of compromise and should be addressed to improve the security posture of the application.

Part B: Technical Report

1. Scope of Testing

The assessment covered the web application interface, including all accessible user functionalities such as authentication, search features, and API endpoints.

2. Methodology

Testing was carried out using a combination of manual techniques and interception tools. The process included:

- Input validation testing
- Request interception and modification
- Payload injection (SQL, XSS)
- Access control verification
- Response analysis

The methodology aligns with standard practices recommended by **OWASP**.

3. Summary of Findings

The following vulnerabilities were identified during testing:

3.1 SQL Injection

- **Endpoint:** Login form
- **Description:** The application does not properly validate user input in the login field, allowing SQL queries to be manipulated.
- **Impact:** Attackers can bypass authentication and gain unauthorized access.

- **Risk Level:** High

3.2 Cross-Site Scripting (XSS)

- **Endpoint:** Search functionality
- **Description:** User input is reflected in the browser without proper encoding, allowing execution of JavaScript.
- **Impact:** Attackers can execute scripts in the user's browser and potentially steal session data.
- **Risk Level:** High

3.3 Broken Access Control (IDOR)

- **Endpoint:** User data API
- **Description:** The application allows users to access other users' data by modifying request parameters.
- **Impact:** Unauthorized exposure of sensitive user information.
- **Risk Level:** High

3.4 Information Disclosure

- **Endpoint:** Error responses
- **Description:** Detailed error messages reveal backend system information.
- **Impact:** Attackers can use this information to plan further attacks.
- **Risk Level:** Medium

Recommendations

To mitigate the identified risks, the following actions are recommended:

- Implement parameterized queries to prevent SQL injection
- Apply proper input validation and output encoding
- Enforce strict access control checks on all endpoints
- Disable detailed error messages in production environments
- Use secure session management practices

Conclusion

The application contains several security weaknesses that could be exploited by attackers. Addressing these issues will significantly reduce the risk of unauthorized access and data exposure. It is recommended that remediation efforts be prioritized based on risk level.